SOFTWARE DESIGN SPECIFICATION

for

Boli.pk

AI-Verified Escrow & Bidding Platform

IEEE/ISO/IEC 1016-2009 Conformant

Version 1.0

Prepared by

Arham Jan & Abdul Qayyum

01-131232-013 & 01-131232-003

Department of Software Engineering | Bahria University, Islamabad

March 2026

Revision History

Version	Date	Description	Authors
1.0	Mar 2026	Initial SDS — implements Boli.pk SRS v2.0. IEEE/ISO/IEC 1016-2009 conformant.	Arham Jan, Abdul Qayyum

Table of Contents

Chapter 1 Introduction 7
1.1 Purpose of this Document 7
1.2 Scope 7
1.3 Relationship to Other Documents 7
1.4 Definitions, Acronyms and Abbreviations 8
Chapter 2 Design Stakeholders and Their Concerns 9
2.1 Stakeholder Identification 9
2.2 Design Concerns by Stakeholder 10
2.2.1 Buyer Concerns 10
2.2.2 Seller Concerns 10
2.2.3 Administrator Concerns 10
2.2.4 SBP / Regulatory Concerns 10
2.2.5 FBR Concerns 11
2.2.6 Development Team Concerns 11
Chapter 3 Design Rationale 12
3.1 ADR-001 — Technology Stack Selection 12
3.2 ADR-002 — Infrastructure and Data Sovereignty 13
3.3 ADR-003 — Event-Driven Architecture with CQRS-Lite 14
3.4 ADR-004 — AI Vetting Pipeline Architecture 15
Chapter 4 Selected Design Viewpoints 16
Chapter 5 Context View 17
5.1 System Boundary 17
5.2 External Actors and Interfaces 17
5.3 System Boundary Constraints 18
Chapter 6 Composition View 19
6.1 Service Decomposition 19
6.1.1 React/Next.js PWA — Frontend Service 19
6.1.2 Go + Centrifugo — Bidding Engine Service 19
6.1.3 Python + FastAPI — AI Vetting Service 20
6.1.4 PostgreSQL + Redis — Data Service 20
6.2 Deployment Architecture 20
6.3 Service Communication Patterns 21
Chapter 7 Logical View — UWE Content Model 22
7.1 Domain Class Diagram 22
7.2 Class Descriptions 23

7.2.1 User 23
7.2.2 Listing 24
7.2.3 Auction 24
7.2.4 Bid 24
7.2.5 Wallet 24
7.2.6 Transaction 24
7.2.7 Escrow 25
7.2.8 MeetupSession 25
7.2.9 TrustBadge 25
7.2.10 Dispute 25
7.2.11 LedgerEntry 25
7.2.12 TaxAccount 25
7.3 Escrow State Machine — Behavioral Model 26
7.4 Auction State Machine — Behavioral Model 28
7.5 Meetup and Escrow State Machine — Behavioral Model 30
Chapter 8 Dependency View — UWE Navigation Model 32
8.1 UWE Navigation Notation 32
8.2 Buyer Navigation Model 33
8.3 Seller Navigation Model 35
8.4 Administrator Navigation Model 36
8.5 State-Driven Navigation Rules 37
Chapter 9 Interface View — UWE Presentation Model and API Contracts 38
9.1 UWE Presentation Model 38
9.1.1 Live Auction Room 38
9.1.2 Meetup IMEI Scan Screen 39
9.1.3 QR Settlement Screen — Seller 40
9.1.4 Wallet Dashboard 41
9.1.5 Listing Creation Form 41
9.1.6 Dispute Arbitration Screen 42
9.2 REST API Contract Summary 42
Chapter 10 Interaction View 44
10.1 Bid Submission Sequence 44
10.2 AI Vetting Pipeline Sequence 44
10.3 Two-Phase Commit Settlement Sequence 45
10.4 Dispute Escalation Sequence 46
10.5 End-of-Day Reconciliation Sequence 46

Chapter 11 Data View 47
11.1 Entity Relationship Diagram 47
11.2 Critical Schema Constraints 47
11.2.1 Financial Integrity Constraints 47
11.2.2 Hash Chain Integrity 47
11.2.3 Immutability and Retention 48
11.2.4 Encryption 48
11.3 Key Table Definitions 48
11.3.1 users table 48
11.3.2 wallets table 49
11.3.3 ledger_entries table 49
Chapter 12 Algorithm View 50
12.1 Luhn Algorithm — IMEI Format Validation 50
12.2 Two-Phase Commit Protocol — Settlement 50
12.3 Saga Pattern — Distributed Compensating Transactions 51
12.4 Circuit Breaker Pattern — External Service Resilience 52
12.5 CQRS-Lite Pattern — Read/Write Separation 52
12.6 AI Vision Model — Device Classification 53
Chapter 13 Resource View 54
13.1 Memory Allocation 54
13.2 CPU Allocation 54
13.3 Database Connection Management 54
13.4 Network Bandwidth Allocation 55
13.5 CEP Resource Profile 55
Chapter 14 Pending Diagrams and Artefacts Register 56

Chapter 1 Introduction

1.1 Purpose of this Document

This Software Design Specification (SDS) defines the complete software design for Boli.pk — an AI-verified escrow and bidding platform for high-value C2C transactions in Pakistan. The document conforms to IEEE/ISO/IEC 1016-2009 and translates the requirements documented in the Boli.pk Software Requirements Specification v2.0 into a set of structured design views that collectively describe the architecture, data structures, interfaces, interactions, algorithms, and resource management strategy of the system.

This document is intended for software engineers who will implement the system, quality assurance engineers who will verify it, and academic evaluators assessing the CEP across Web Engineering, Cloud Computing, Artificial Intelligence, Software Quality Assurance, Human Computer Interaction, and Technical Writing and Presentation Skills courses.

1.2 Scope

This SDS covers the complete design of the Boli.pk MVP as defined in SRS v2.0. It documents twelve design views organized under nine viewpoints as defined by IEEE 1016-2009. All design views are consistent with and traceable to the requirements in the SRS. Where the SRS defines what the system must do, this SDS defines how the system is structured to do it.

This document does not replace the SRS. It augments it. Any conflict between this SDS and the SRS must be resolved in favour of the SRS.

1.3 Relationship to Other Documents

Document	Relationship
Boli.pk SRS v2.0	Parent requirements document. This SDS implements all requirements defined therein.
Boli.pk ADR-001	Technology stack selection — incorporated as Design Rationale in Chapter 3.
Boli.pk ADR-002	Infrastructure and data sovereignty — incorporated as Design Rationale in Chapter 3.
Boli.pk ADR-003	EDA with CQRS-Lite — incorporated as Design Rationale and Architectural View.

Boli.pk ADR-004	AI vetting pipeline — incorporated as Design Rationale and Algorithm View.
Boli.pk Use Case Specification	Companion document describing user-system interactions. Referenced by Interface View.

1.4 Definitions, Acronyms and Abbreviations

All terms, acronyms, and abbreviations carry the same definitions as specified in Boli.pk SRS v2.0, Chapter 1, Sections 1.4 and 1.5. Additional design-specific terms used in this document:

Term	Definition
Viewpoint	A specification of the conventions for constructing and using a design view. Defined in IEEE 1016-2009.
Design View	A representation of a whole system from the perspective of a related set of concerns. Defined in IEEE 1016-2009.
Design Element	A named design constituent — a module, component, class, interface, or relationship.
CQRS-Lite	Command Query Responsibility Segregation (simplified) — read path uses Redis cache, write path uses PostgreSQL.
EDA	Event-Driven Architecture — the bidding engine processes bids as events through Redis Streams.
Saga Pattern	A distributed transaction pattern using compensating transactions to maintain consistency across microservices.
Circuit Breaker	A design pattern that prevents cascade failures by stopping calls to a failing service and queuing operations.
Dead Letter Queue (DLQ)	A queue that receives messages that cannot be processed after the maximum retry count is exceeded.
Two-Phase Commit (2PC)	A distributed commit protocol ensuring all participants in a transaction either all commit or all abort.
Luhn Algorithm	A checksum formula used to validate IMEI numbers (ISO/IEC 7812).

Chapter 2 Design Stakeholders and Their Concerns

IEEE 1016-2009 requires the SDS to explicitly identify design stakeholders and document the design concerns of each. This chapter fulfils that requirement for Boli.pk. Design concerns are distinct from functional requirements — they describe the aspects of the design that each stakeholder needs to understand or evaluate.

2.1 Stakeholder Identification

Stakeholder	Role in Relation to Boli.pk
Buyers	End users who fund wallets, place bids, coordinate meetups, and scan QR codes to release escrow
Sellers	End users who create listings, submit AI vetting assets, manage auctions, and generate settlement QR codes
Platform Administrators	Boli.pk operations staff managing disputes, reviewing flagged listings, and enforcing penalties
Development Team	Engineers implementing, testing, and deploying the system
SBP (State Bank of Pakistan)	Regulator overseeing financial data sovereignty, AML compliance, and EMI licensing requirements
FBR (Federal Board of Revenue)	Regulator requiring WHT collection, NTN verification, and 5-year audit trail retention
PTA (Pakistan Telecommunication Authority)	Regulator governing IMEI/DIRBS device verification access
Academic Evaluators	Panel of professors and associate professors evaluating the system across six courses
Future Investors	Parties evaluating the platform for funding based on technical architecture and regulatory readiness

2.2 Design Concerns by Stakeholder

2.2.1 Buyer Concerns

- Transaction Safety: The system must never release their funds without explicit consent through the QR scan flow.
- IMEI Verification: The system must verify the physical device matches the listing before unlocking the QR, protecting them from device swap fraud.
- Bid Latency: The real-time bidding interface must reflect their bid immediately without perceptible delay.
- Refund Assurance: If the seller ghosts or the meetup fails, their funds must return automatically and completely.
- Transparency: The total amount they will pay including fees must be visible before they commit to a bid.

2.2.2 Seller Concerns

- Payment Guarantee: Once the item is handed over and QR scanned, funds must arrive in their wallet without delay.
- Listing Integrity: The AI pipeline must not reject legitimate listings through false positives.
- Non-Repudiation: The cryptographic settlement receipt must be sufficient evidence to counter any fraudulent reversal claim.
- PTA Grey Market: Unregistered devices must be listable with disclosure rather than rejected outright.

2.2.3 Administrator Concerns

- Evidence Preservation: All dispute evidence must be frozen and immutable at the moment a dispute is raised.
- Maker-Checker: No single administrator must be able to unilaterally move user funds.
- Audit Trail: Every financial state change must be logged with sufficient detail for forensic reconstruction.
- Fraud Detection: Shill bidding and AML velocity patterns must be surfaced automatically for review.

2.2.4 SBP / Regulatory Concerns

- Data Sovereignty: All PII and financial data must remain on physically Pakistani infrastructure.
- AML Compliance: Transaction velocity monitoring and account freezing must be automated.
- Ledger Integrity: The double-entry ledger must be mathematically provable at all times.
- EMI Licensing: The architecture must be documented to demonstrate readiness for future EMI license application.

2.2.5 FBR Concerns

- WHT Automation: 1% withholding tax must be automatically deducted and held in a segregated account.
- NTN Enforcement: Sellers above the Rs. 100,000 threshold must have verified NTNs before listing.
- 5-Year Retention: All transaction records must be retained and retrievable for 5 years minimum.

2.2.6 Development Team Concerns

- Modularity: Services must be independently deployable and testable.
- Swappable Dependencies: Mock and production external services must share identical interfaces.
- Test Coverage: Financial ledger code paths must achieve 100% automated test coverage.
- Local Deployment: The entire system must run offline via docker-compose for development and CEP demonstration.

Chapter 3 Design Rationale

IEEE 1016-2009 requires design rationale to be documented within the SDS. The following section formally incorporates the four Architecture Decision Records produced for Boli.pk. Each ADR documents the design context, the decision made, the alternatives considered, and the consequences accepted. These records are permanent — if a decision is reversed, a new ADR supersedes this section.

3.1 ADR-001 — Technology Stack Selection

Attribute	Detail
Status	ACCEPTED — March 2026
Context	Boli.pk requires three simultaneously operational workloads: real-time auction bidding (sub-second latency), financial escrow operations (ACID compliance), and AI-powered listing verification (compute-intensive). These workloads have incompatible performance profiles and cannot be optimally served by a single technology.
Decision	Go + Centrifugo (bidding engine), Python + FastAPI (AI service), PostgreSQL + Redis (ledger and cache), React/Next.js PWA (frontend), GitHub Actions (CI/CD).

Alternatives Considered and Rejected:

Alternative	Decision
MongoDB instead of PostgreSQL	REJECTED
Node.js instead of Go	REJECTED
Django/Flask instead of FastAPI	REJECTED
Monolithic architecture	REJECTED
React Native instead of PWA	REJECTED

Consequences:

- The three-service architecture requires defined API contracts between every service pair before implementation begins.
- Go expertise must be developed by the team. Centrifugo configuration adds complexity. Cursor AI-assisted coding mitigates the learning curve.
- Redis is a cache only — it is never the source of truth for any financial value. This must be enforced in code review.

3.2 ADR-002 — Infrastructure and Data Sovereignty

Attribute	Detail
Status	ACCEPTED — March 2026
Context	The original SRS contained three mutually incompatible constraints: RPO 0 seconds, P99 write latency under 15ms, and strict data sovereignty on Pakistani infrastructure. These cannot be simultaneously satisfied. A tiered model was adopted.
Decision	Financial and PII data: Pakistani data centers (PTCL/Nayatel/TPS), asynchronous streaming replication, RPO 5 seconds, RTO under 5 minutes, uptime 99.5%. Non-financial data: international cloud, synchronous replication, RPO 0 seconds. 15ms latency target applies to Redis bid cache only. Financial PostgreSQL writes: 100-200ms acceptable.

Rationale for RPO 5 Seconds:

RPO 0 requires synchronous replication which adds 20-50ms per write on Pakistani infrastructure, violating the 15ms cache latency requirement. Redis Streams persistence provides an additional recovery layer — bid events buffered in the stream can reconstruct up to 5 seconds of lost PostgreSQL writes on failover.

Rationale for 99.5% Uptime:

Pakistani data center providers cannot contractually guarantee Tier-1 cloud SLAs. The original 99.99% target was aspirational and architecturally unachievable with available Pakistani infrastructure. 99.5% reflects realistic SLA capability.

Consequences:

- The platform accepts theoretical maximum data loss of 5 seconds in a catastrophic failure. This is disclosed in the platform's terms of service.
- Non-financial data (images, logs, search indexes) may use AWS Bahrain or Mumbai for superior availability.
- Geographic expansion beyond Pakistan requires a new ADR addressing cross-border data transfer compliance.

3.3 ADR-003 — Event-Driven Architecture with CQRS-Lite

Attribute	Detail
Status	ACCEPTED — March 2026
Context	The bidding engine must handle thousands of concurrent bid submissions. A synchronous architecture serialises all bids through the PostgreSQL write path, creating a bottleneck under load. EDA decouples bid acceptance from bid persistence.
Decision	EDA with CQRS-Lite applies to the bidding engine only. Write path: bids written to Redis Streams immediately, acknowledged to bidder, consumed asynchronously by Go worker, committed to PostgreSQL with row-level locking. Read path: auction state read from Redis cache (sub-15ms). Escrow path: all financial operations use synchronous PostgreSQL transactions only — never queued.

Known Risks and Mitigations:

Risk	Mitigation
Phantom Win — bidder sees winning but loses on DB commit	Optimistic UI shows Pending state until WebSocket confirms DB write. UI never shows confirmed win until PostgreSQL acknowledges.
Poison Pill — malformed event blocks stream consumer	Dead Letter Queue captures events failing after 3 retries. Consumer skips to next event. DLQ events reviewed manually.
Stream backpressure — consumer falls behind under load	Maximum stream length capped. Streams older than auction duration trimmed. Consumer lag monitored with 500ms alert threshold.
At-least-once delivery — same bid processed twice	Cryptographic idempotency key checked before each PostgreSQL write. Duplicate events silently discarded.
Redis failure during active auction	Redis AOF persistence enabled. Stream replayed from last acknowledged position on restart. Auction paused during recovery.

Operations Explicitly Outside EDA Scope:

- Escrow fund locking after auction close
- QR-triggered fund release
- Refund processing on IMEI mismatch or meetup timeout
- Tax withholding and platform fee deduction at settlement
- Dispute fund freezing

3.4 ADR-004 — AI Vetting Pipeline Architecture

Attribute	Detail
Status	ACCEPTED — March 2026
Context	The original pipeline design (ImageHash + EXIF extraction + NLP price validation) was found fundamentally inadequate: ImageHash detects duplicate photos but does not authenticate items; EXIF metadata is trivially spoofed; NLP price validation has no reliable Pakistani market data source. Complete redesign required.
Decision	Two-phase pipeline. Phase 1: three deterministic hard gates (fail any = immediate rejection). Phase 2: three probabilistic scoring checks (scores feed trust badge). PTA unregistered devices: disclosure badge, not rejection. Trust score displayed as badge only — raw score is internal.

Phase 1 — Deterministic Hard Gates:

Gate	Check
Gate 1	IMEI Format (Luhn)
Gate 2	PTA DIRBS Lookup
Gate 3	TAC Model Match

Phase 2 — Probabilistic Scoring Checks:

Check	Methodology
Check 4: Image Consistency	Vision model verifies device in photos matches stated make/model. Catches camera module count mismatches, form factor discrepancies.
Check 5: Condition vs Photos	Model compares stated condition rating against visible damage indicators in photos (screen, frame, camera lens).
Check 6: Price Sanity	Stated price compared against scraped market data from OLX, PriceOye, Zmobiles. Price >35% below market median → manual review escalation regardless of score.

Trust Badge Classification:

Score Range	Badge
80-100	VERIFIED
50-79	REVIEWED
Below 50	PENDING_REVIEW
Phase 1 failure	REJECTED

Chapter 4 Selected Design Viewpoints

IEEE 1016-2009 requires the SDS to identify which design viewpoints have been selected and explain why. The following nine viewpoints have been selected for Boli.pk. Together they address all design stakeholder concerns identified in Chapter 2.

Viewpoint	Design Concern Addressed
Context Viewpoint	How Boli.pk fits within its external environment — external actors, system boundaries, and integration points
Composition Viewpoint	How the system is decomposed into components and services — the microservice architecture
Logical Viewpoint (UWE Content Model)	Domain object structure, class relationships, and state machine behavior — the information architecture
Dependency Viewpoint (UWE Navigation Model)	Navigation architecture — how users traverse the application and how screens are connected
Interface Viewpoint (UWE Presentation Model)	Screen structure, interaction design, and API contract definitions
Interaction Viewpoint	How objects and services communicate over time — sequence diagrams for critical flows
Data Viewpoint	Persistent data structure — entity relationships, schema design, and integrity constraints
Algorithm Viewpoint	Key algorithms and design patterns applied — Luhn, 2PC, Saga, Circuit Breaker, CQRS-Lite
Resource Viewpoint	How system resources are allocated and managed — memory, connections, CPU

Chapter 5 Context View

The context view describes Boli.pk's position within its external environment. It identifies all external actors, the boundaries of the system, and the nature of each external interface. This view addresses the design concerns of all stakeholders regarding system scope and integration points.

5.1 System Boundary

Boli.pk is a self-contained platform. The system boundary encompasses four internal services: the React/Next.js PWA, the Go bidding engine, the Python AI microservice, and the PostgreSQL/Redis data layer. Everything outside this boundary — payment gateways, government databases, device hardware, notification services — is an external actor.

[PLACEHOLDER]

Figure 5.1 — System Context Diagram: Boli.pk and all external actors with interface types. To be produced in draw.io or PlantUML and inserted here.

5.2 External Actors and Interfaces

External Actor	Interface Type
Buyer (mobile browser)	HTTPS + WSS
Seller (mobile browser)	HTTPS + WSS
Administrator (web browser)	HTTPS
PTA DIRBS	REST API — IMEI lookup
GSMA TAC Database	REST API — TAC model lookup
FBR NTN Validation	REST API — NTN format and filer status
Push Notification Service	Native push protocol
Payment Gateway (Raast)	REST API — wallet funding/withdrawal

Device Hardware (camera, biometric, GPS)	Web APIs (getUserMedia, WebAuthn, Geolocation)

5.3 System Boundary Constraints

- All PII and financial data processing must occur within the system boundary on Pakistani-hosted infrastructure (ADR-002).
- No financial value may leave or enter the system boundary through any pathway other than the designated wallet top-up and withdrawal interfaces.
- All mock external interfaces implement the identical JSON contract as their production counterparts, enabling production activation through configuration change only.

Chapter 6 Composition View

The composition view describes how Boli.pk is decomposed into components, services, and modules. It addresses the development team's concern for modularity and independent deployability, and the SBP's concern for clear architectural boundaries between financial and non-financial processing.

6.1 Service Decomposition

[PLACEHOLDER]

Figure 6.1 — High-Level Component Diagram: Four services, their internal modules, and inter-service communication pathways. To be produced and inserted here.

Boli.pk is composed of four independently deployable services:

6.1.1 React/Next.js PWA — Frontend Service

Responsible for all user-facing interfaces across buyer, seller, and administrator roles. Communicates with the Go bidding engine via WSS for real-time auction state and with the backend via REST for all other operations.

- Auction Browser Module — marketplace feed, listing detail, trust badge display
- Bidding Module — live auction room, real-time bid state, optimistic UI
- Wallet Module — balance display, top-up interface, transaction history
- Listing Creation Module — seller form, image upload, AI vetting status
- Meetup Module — coordination interface, IMEI scanner, QR display/scanner
- Dispute Module — dispute initiation, arbitration chat, evidence upload
- Admin Module — listing review queue, dispute resolution, penalty management
- Authentication Module — KYC onboarding, NTN verification, session management

6.1.2 Go + Centrifugo — Bidding Engine Service

Responsible for all real-time auction operations. Manages WebSocket connections via Centrifugo, processes bids as events through Redis Streams, and coordinates auction lifecycle events.

- WebSocket Gateway — connection management, heartbeat, reconnection handling
- Bid Validator — proof-of-funds check, shill detection, rate limiting
- Redis Stream Producer — writes validated bids to stream with idempotency key
- Redis Stream Consumer — reads bids from stream, commits to PostgreSQL with row-level locking
- Auction Lifecycle Manager — start, close, winner determination, loser notification

- Dead Letter Queue Handler — processes failed bid events after retry exhaustion

6.1.3 Python + FastAPI — AI Vetting Service

Responsible for all listing verification operations. Executes the six-check pipeline on submitted listing assets and returns structured badge classifications. Runs as a fully isolated asynchronous service with no shared state with other services.

- IMEI Format Validator — Luhn algorithm checksum verification
- DIRBS Interface — mock/live PTA DIRBS lookup client
- TAC Resolver — GSMA TAC database lookup client
- Vision Model Wrapper — pre-trained device classification model inference
- Condition Assessor — image analysis for damage indicator detection
- Price Sanity Checker — market data scraper and comparison engine
- Badge Classifier — aggregates all check results into final badge classification

6.1.4 PostgreSQL + Redis — Data Service

The financial backbone of the platform. PostgreSQL serves as the immutable source of truth for all financial state. Redis serves as the high-velocity cache for auction state during live auctions only.

- Financial Ledger Schema — double-entry tables with append-only constraints
- Escrow State Machine Tables — 13-state money lifecycle with transition enforcement
- KYC and NTN Registry — encrypted PII storage with role-based access
- Audit Trail Tables — hash-chained append-only transaction log
- Tax Segregation Accounts — three isolated system wallets (WHT, ICT Sales Tax, Revenue)
- Redis Bid Cache — live auction state with TTL-based eviction
- Redis Streams — EDA event bus for bid processing pipeline

6.2 Deployment Architecture

[PLACEHOLDER]

Figure 6.2 — Deployment Diagram: Physical infrastructure topology showing Pakistani DC nodes, cloud nodes, and service placement. To be produced and inserted here.

The deployment follows the tiered data sovereignty model defined in ADR-002:

Pakistani Data Center (Primary):

- PostgreSQL primary node — all financial ledger, KYC, and escrow data
- PostgreSQL replica node — asynchronous streaming replication, RPO 5 seconds

- Go bidding engine instances — co-located with Redis for minimum cache latency
- Redis primary node — bid cache and Streams bus

International Cloud (AWS Bahrain or Mumbai):

- Object storage — listing images and media files
- Application logs and monitoring data
- Search indexes for marketplace discovery
- CDN for static PWA assets

CEP Local Deployment (docker-compose):

- All four services run as Docker containers on a single machine
- All external dependencies fulfilled by mock services in the same compose network
- No external network access required

[PLACEHOLDER]

Figure 6.3 — Docker Compose Architecture: Container definitions, networks, and volumes for CEP offline deployment. To be produced and inserted here.

6.3 Service Communication Patterns

Source	Destination
PWA	Go Bidding Engine
PWA	Go Bidding Engine
PWA	Python AI Service
Go Engine	PostgreSQL
Go Engine	Redis
Go Worker	Redis Streams
Go Worker	PostgreSQL
AI Service	Mock DIRBS
AI Service	Mock TAC DB
All Services	SIEM

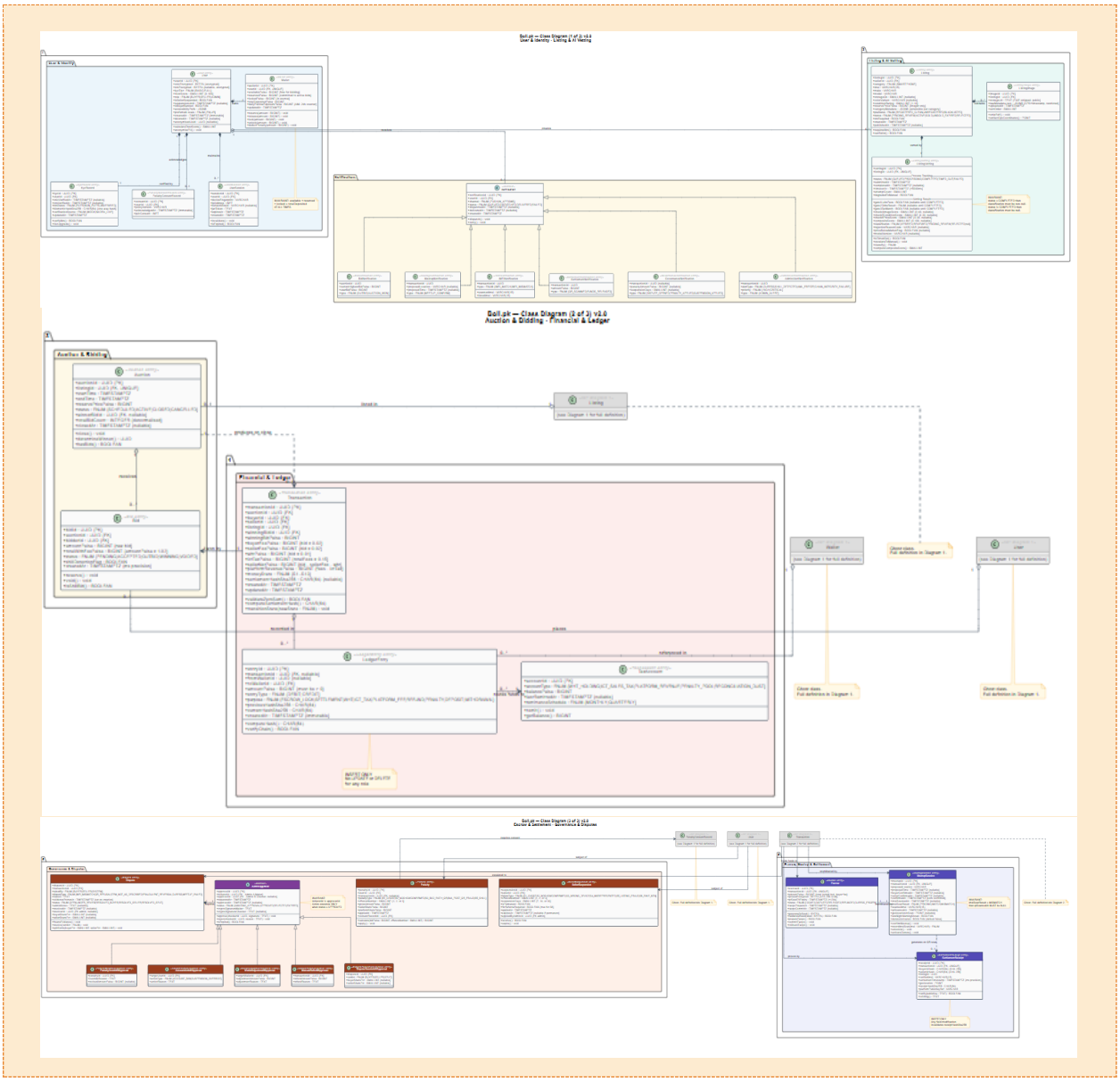
Chapter 7 Logical View — UWE Content Model

The logical view describes the domain object structure of Boli.pk using a UML class diagram. This view corresponds to the UWE Content Model required for Assignment 2. It addresses the design concerns of the development team (object structure), SQA evaluators (testable domain model), and the Web Engineering course (UWE methodology).

7.1 Domain Class Diagram

The following class diagram defines all primary domain objects, their key attributes, and the associations between them. Multiplicities and association types are shown. This diagram forms the basis of the database schema defined in Chapter 11.

(SEE NEXT PAGE)



7.2 Class Descriptions

7.2.1 User

Represents both buyers and sellers. A user may act as a buyer in some transactions and a seller in others. Key attributes: userId (UUID), cnic (encrypted string), ntn (encrypted string, nullable), kycTier (BASIC|FULL), trustScore (integer 0-100), role (BUYER|SELLER|ADMIN), createdAt (timestamp).

- A User owns exactly one Wallet.
- A User may create zero or more Listings.
- A User may place zero or more Bids.
- A User may be subject to zero or more Penalties.

7.2.2 Listing

Represents an item submitted for auction. Key attributes: listingId (UUID), sellerId (UUID FK), imei (string, required for smartphones), make (string), model (string), condition (integer 1-10), askingPricePaisa (integer), status (PENDING_REVIEW|VERIFIED|REVIEWED|REJECTED|ACTIVE|SOLD|UNSOLD_EXPIRED), createdAt (timestamp).

- A Listing belongs to exactly one User (seller).
- A Listing has exactly one TrustBadge assigned after AI vetting.
- A Listing has at most one active Auction.

7.2.3 Auction

Represents a live bidding event for a listing. Key attributes: auctionId (UUID), listingId (UUID FK), startTime (timestamp), endTime (timestamp), reservePricePaisa (integer), status (SCHEDULED|ACTIVE|CLOSED|CANCELLED), winnerBidId (UUID FK nullable).

- An Auction belongs to exactly one Listing.
- An Auction may have zero or more Bids.
- A closed Auction with bids produces exactly one Transaction.

7.2.4 Bid

Represents a single bid placed by a buyer. Key attributes: bidId (UUID), auctionId (UUID FK), bidderId (UUID FK), amountPaisa (integer), totalWithFeePaisa (integer — amountPaisa × 1.02), status (PENDING|ACCEPTED|OUTBID|WINNING|VOIDED), idempotencyKey (UUID), createdAt (timestamp with millisecond precision).

- A Bid belongs to exactly one Auction.
- A Bid belongs to exactly one User (bidder).
- A Bid transitions through status states in strict sequence.

7.2.5 Wallet

Represents a user's financial balance. Key attributes: walletId (UUID), userId (UUID FK), availablePaisa (integer), reservedPaisa (integer), lockedPaisa (integer), totalDepositedPaisa (integer). Invariant: availablePaisa + reservedPaisa + lockedPaisa = totalDepositedPaisa at all times.

7.2.6 Transaction

Represents a completed or in-progress transaction between a buyer and seller. Key attributes: transactionId (UUID), auctionId (UUID FK), buyerId (UUID FK), sellerId (UUID FK), listingId (UUID FK), moneyState (S1 through S13 enumeration), winningBidPaisa (integer), settlementHashSHA256 (string nullable), createdAt (timestamp), updatedAt (timestamp).

7.2.7 Escrow

Represents the locked funds for an active transaction. Key attributes: escrowId (UUID), transactionId (UUID FK), amountPaisa (integer), qrSeedEncrypted (string), qrSeedTTLExpiry (timestamp), imeiVerified (boolean), status (LOCKED|DISPUTED|RELEASED|REFUNDED).

7.2.8 MeetupSession

Represents the structured meetup coordination between buyer and seller. Key attributes: meetupId (UUID), transactionId (UUID FK), confirmedAt (timestamp nullable), proposedLocation (string), proposedTime (timestamp nullable), imeiScannedAt (timestamp nullable), imeiMatchResult (MATCH|MISMATCH|PENDING), qrScannedAt (timestamp nullable).

7.2.9 TrustBadge

Represents the AI vetting result for a listing. Key attributes: badgeId (UUID), listingId (UUID FK), gate1Result (PASS|FAIL), gate2Result (REGISTERED_CLEAN|UNREGISTERED|BLACKLISTED), gate3Result (PASS|FAIL), check4Score (integer 0-40), check5Score (integer 0-30), check6Score (integer 0-30), compositeScore (integer 0-100), classification (VERIFIED|REVIEWED|PENDING_REVIEW|REJECTED), rejectionReason (string nullable).

7.2.10 Dispute

Represents an active dispute. Key attributes: disputeId (UUID), transactionId (UUID FK), raisedBy (BUYER|SELLER|SYSTEM), reason (string), evidenceFrozenAt (timestamp), status (OPEN|UNDER_REVIEW|RESOLVED), adminResolution (string nullable), resolvedAt (timestamp nullable).

7.2.11 LedgerEntry

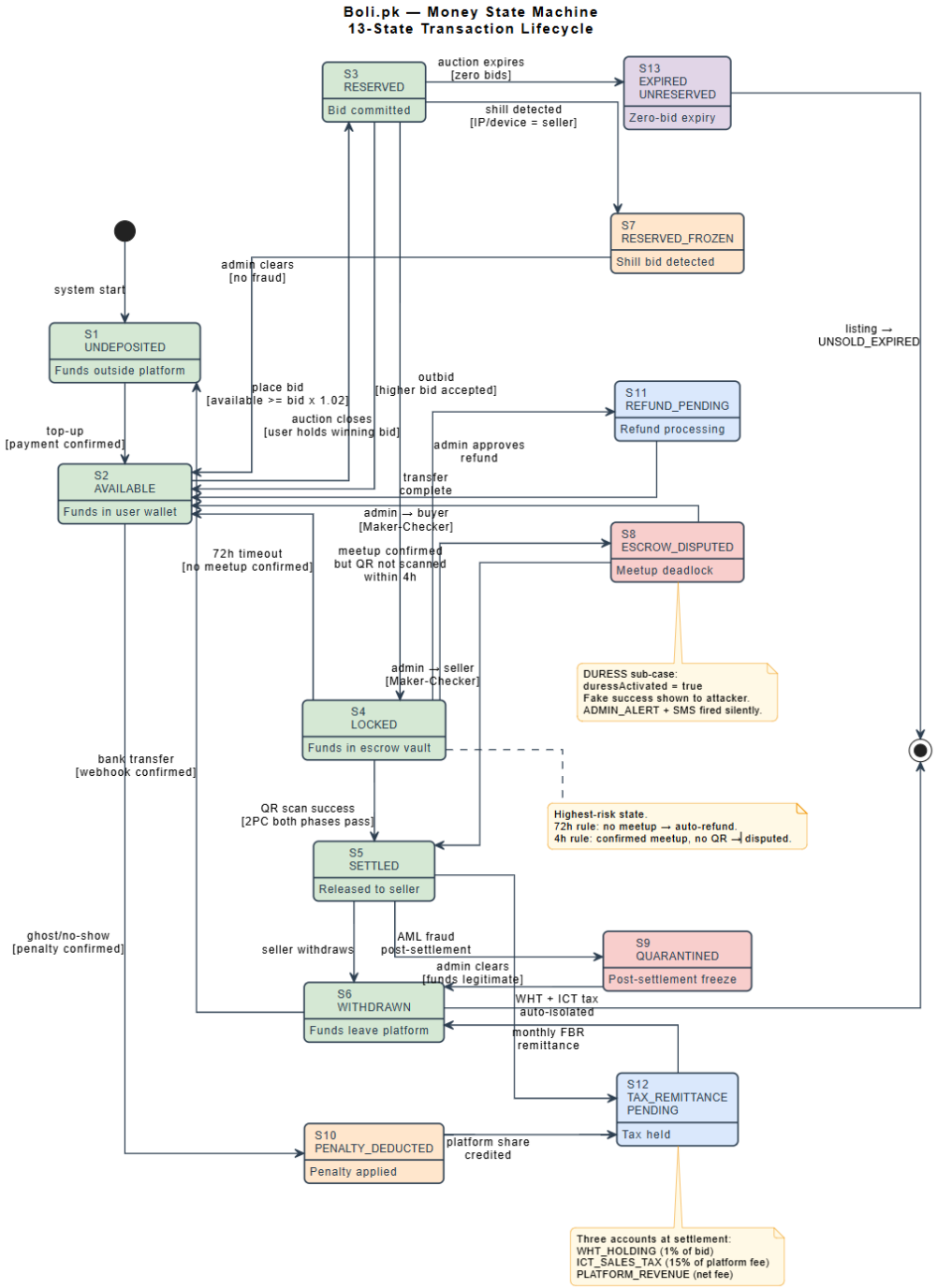
Represents a single immutable financial record. Key attributes: entryId (UUID), transactionId (UUID FK nullable), fromWalletId (UUID FK nullable), toWalletId (UUID FK), amountPaisa (integer), entryType (DEBIT|CREDIT), purpose (enumeration), previousHashSHA256 (string), currentHashSHA256 (string), createdAt (timestamp).

7.2.12 TaxAccount

Represents one of three segregated system accounts. Key attributes: accountId (UUID), accountType (WHT_HOLDING|ICT_SALES_TAX_HOLDING|PLATFORM_REVENUE|PENALTY_POOL|RECONCILIATION), balancePaisa (integer), lastRemittedAt (timestamp nullable).

7.3 Escrow State Machine — Behavioral Model

The following state machine diagram documents the 13-state money lifecycle defined in SRS DR-03. This corresponds to the UWE behavioral/state diagram component of the Content Model

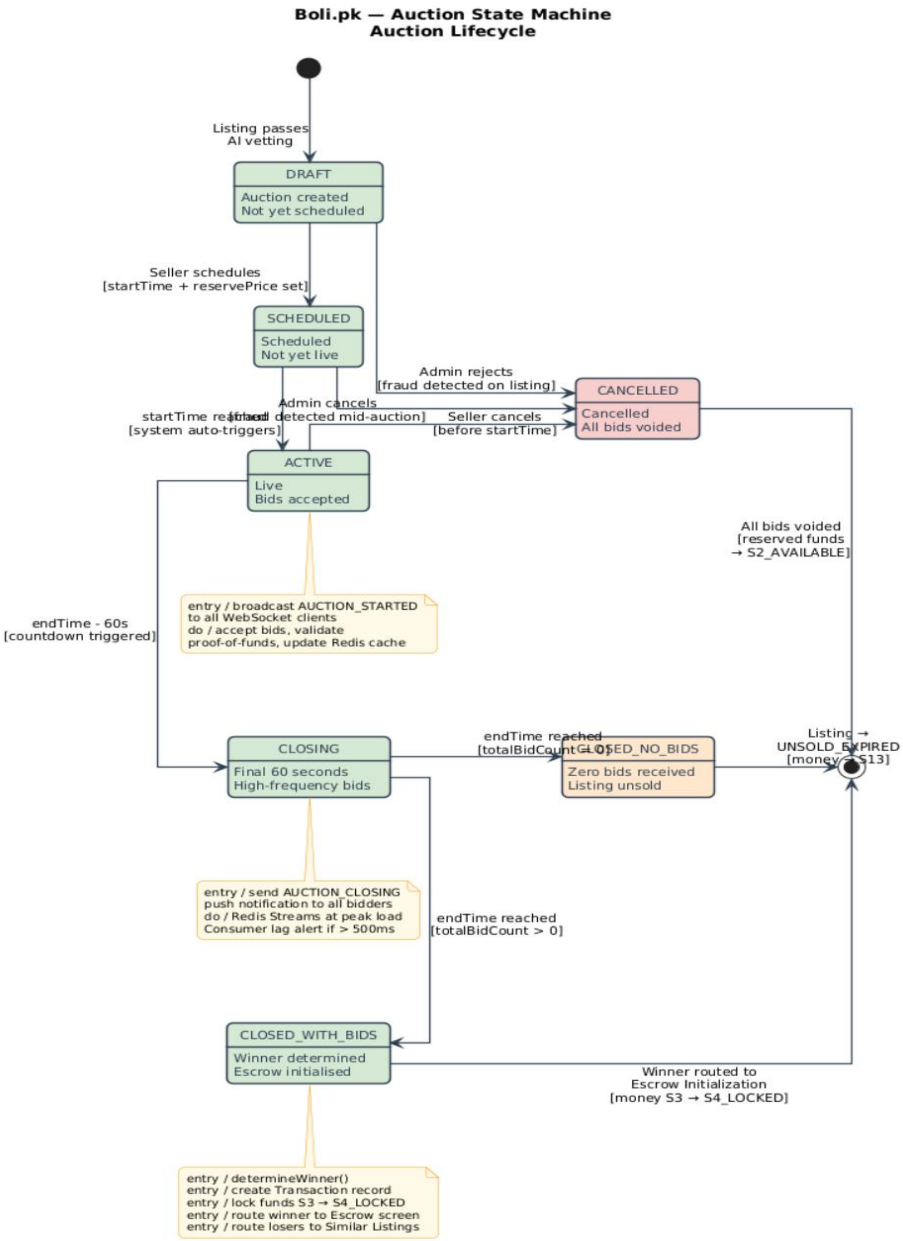


State Transition Summary:

From State	To State
S1 UNDEPOSITED	S2 AVAILABLE
S2 AVAILABLE	S3 RESERVED
S3 RESERVED	S2 AVAILABLE
S3 RESERVED	S7 RESERVED-FROZEN
S3 RESERVED	S4 LOCKED
S4 LOCKED	S2 AVAILABLE
S4 LOCKED	S8 ESCROW-DISPUTED
S4 LOCKED	S5 SETTLED
S5 SETTLED	S9 QUARANTINED
S5 SETTLED	S6 WITHDRAWN
S6 WITHDRAWN	S1 UNDEPOSITED
S7 RESERVED-FROZEN	S2 AVAILABLE
S8 ESCROW-DISPUTED	S2 AVAILABLE
S8 ESCROW-DISPUTED	S5 SETTLED
S9 QUARANTINED	S6 WITHDRAWN
S2 AVAILABLE	S10 PENALTY-DEDUCTED
S10 PENALTY-DEDUCTED	S12 TAX-REMITTANCE-PENDING
S4 LOCKED	S11 REFUND-PENDING
S11 REFUND-PENDING	S2 AVAILABLE
S5 SETTLED	S12 TAX-REMITTANCE-PENDING
S12 TAX-REMITTANCE-PENDING	S6 WITHDRAWN
S3 RESERVED	S13 EXPIRED-UNRESERVED

7.4 Auction State Machine — Behavioral Model

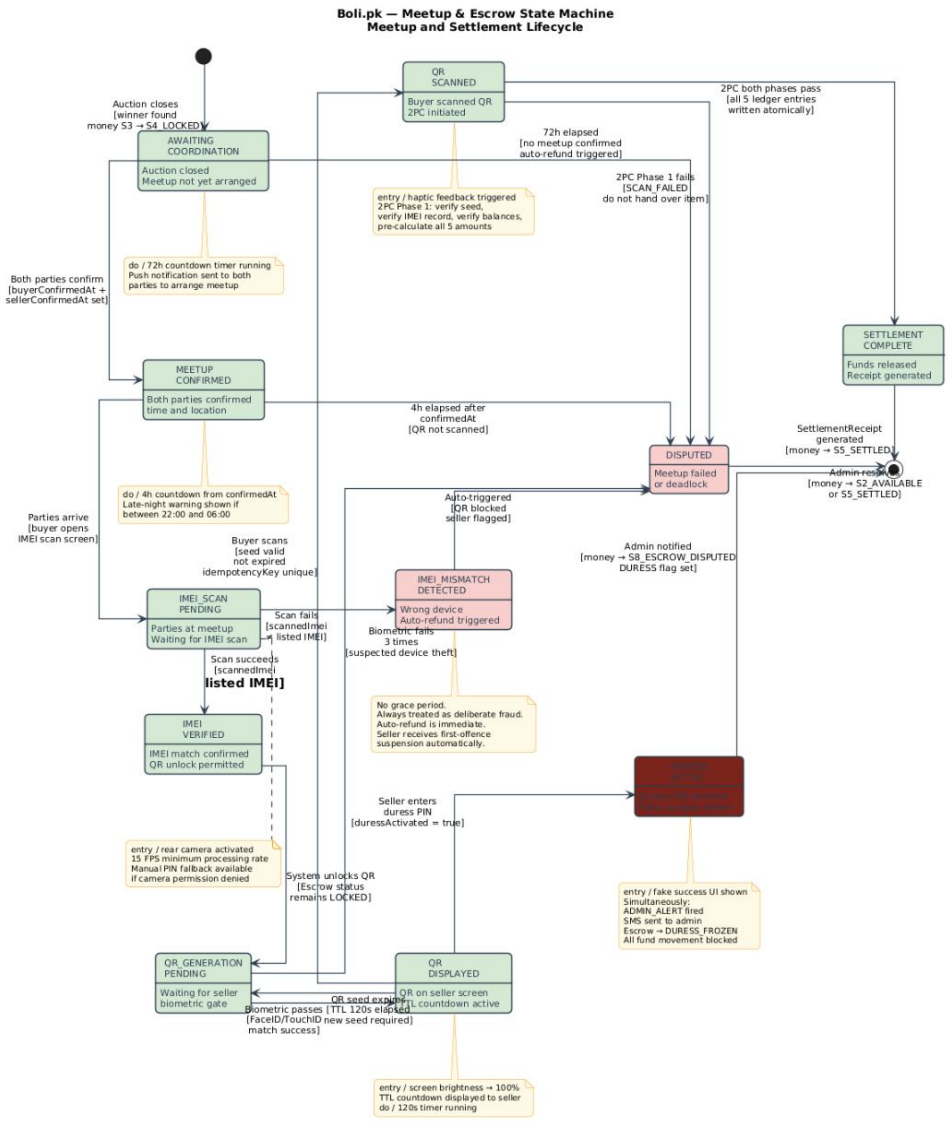
The following state machine diagram documents the auction lifecycle for Boli.pk. An auction is created when a listing passes AI vetting and the seller schedules a start time. The auction progresses through scheduling, live bidding, and a closing window before terminating in one of three possible outcomes — a winner is determined, no bids were received, or the auction was cancelled. This state machine is directly referenced by the money state machine — the CLOSED_WITH_BIDS state triggers the money transition from S3_RESERVED to S4_LOCKED, and the CLOSED_NO_BIDS state triggers S13_EXPIRED_UNRESERVED.



From State	To State

7.5 Meetup and Escrow State Machine — Behavioral Model

The following state machine diagram documents the meetup coordination and cryptographic settlement lifecycle. This state machine activates immediately when an auction closes with a winner and the money transitions to S4_LOCKED. It governs the physical meetup coordination, IMEI verification, QR code generation and scanning, and the Two-Phase Commit settlement protocol. The SETTLEMENT_COMPLETE state corresponds to the money transition from S4_LOCKED to S5_SETTLED. The DISPUTED state corresponds to the money transition to S8_ESCROW_DISPUTED. The DURESS_ACTIVE state sets the DURESS flag on S8_ESCROW_DISPUTED.



From State	To State
S1 UNDEPOSITED	S2 AVAILABLE
S2 AVAILABLE	S3 RESERVED
S3 RESERVED	S2 AVAILABLE
S3 RESERVED	S7 RESERVED-FROZEN
S3 RESERVED	S4 LOCKED
S4 LOCKED	S2 AVAILABLE
S4 LOCKED	S8 ESCROW-DISPUTED
S4 LOCKED	S5 SETTLED
S5 SETTLED	S9 QUARANTINED
S5 SETTLED	S6 WITHDRAWN
S6 WITHDRAWN	S1 UNDEPOSITED
S7 RESERVED-FROZEN	S2 AVAILABLE
S8 ESCROW-DISPUTED	S2 AVAILABLE
S8 ESCROW-DISPUTED	S5 SETTLED
S9 QUARANTINED	S6 WITHDRAWN
S2 AVAILABLE	S10 PENALTY-DEDUCTED
S10 PENALTY-DEDUCTED	S12 TAX-REMITTANCE-PENDING
S4 LOCKED	S11 REFUND-PENDING
S11 REFUND-PENDING	S2 AVAILABLE
S5 SETTLED	S12 TAX-REMITTANCE-PENDING
S12 TAX-REMITTANCE-PENDING	S6 WITHDRAWN
S3 RESERVED	S13 EXPIRED-UNRESERVED

Chapter 8 Dependency View — UWE Navigation Model

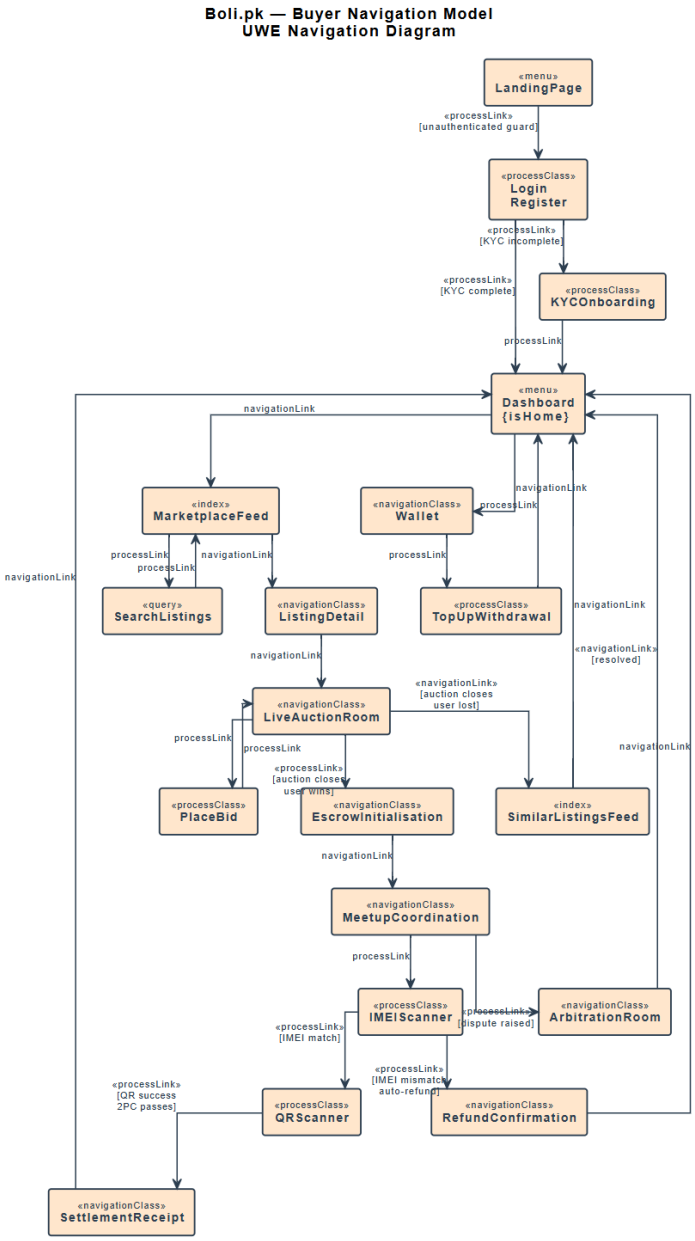
The dependency view describes how users navigate through the Boli.pk application. Using UWE notation, navigation classes represent screens, navigation links represent directional navigation paths, and process links represent actions that trigger processes. This view directly satisfies the Navigation Model requirement of Assignment 2 and addresses the HCI course's concern for user flow design.

8.1 UWE Navigation Notation

UWE Element	Notation
«navigationClass»	Rounded rectangle
«navigationLink»	Directed arrow
«processLink»	Dashed directed arrow
{isHome}	Tag on navigationClass
«anchor»	Small circle on screen

8.2 Buyer Navigation Model

The buyer navigation model covers the complete flow from unauthenticated landing through auction participation, meetup coordination, and settlement.



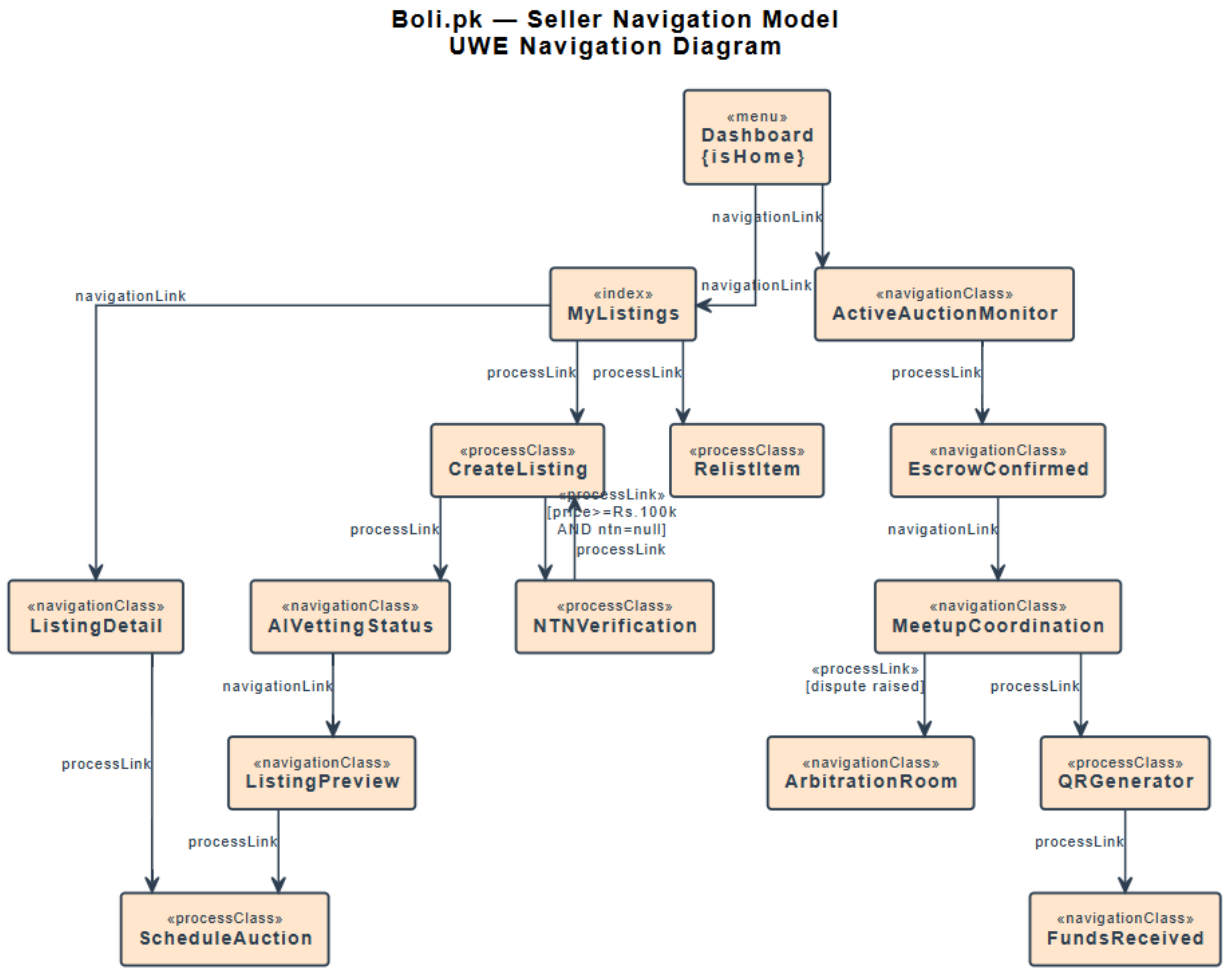
Key Navigation Paths for Buyer Flow:

- Landing Page → [unauthenticated guard] → Login/Register → [KYC flow if incomplete] → Dashboard {isHome}
- Dashboard → Marketplace Feed → Listing Detail → [bid action — processLink] → Live Auction Room
- Live Auction Room → [auction close, winner — processLink] → Escrow Initialization → Meetup Coordination
- Meetup Coordination → IMEI Scanner → [IMEI match — processLink] → QR Scanner → [QR success — processLink] → Settlement Receipt → Dashboard
- Meetup Coordination → [IMEI mismatch — processLink] → Refund Confirmation → Dashboard

- Meetup Coordination → [dispute raised] → Arbitration Room → [resolved] → Dashboard
- Live Auction Room → [auction close, loser] → Similar Listings Feed → Dashboard
- Dashboard → Wallet → Top-up / Withdrawal → Dashboard

8.3 Seller Navigation Model

The seller navigation model covers listing creation, auction management, and meetup settlement from the seller's perspective.



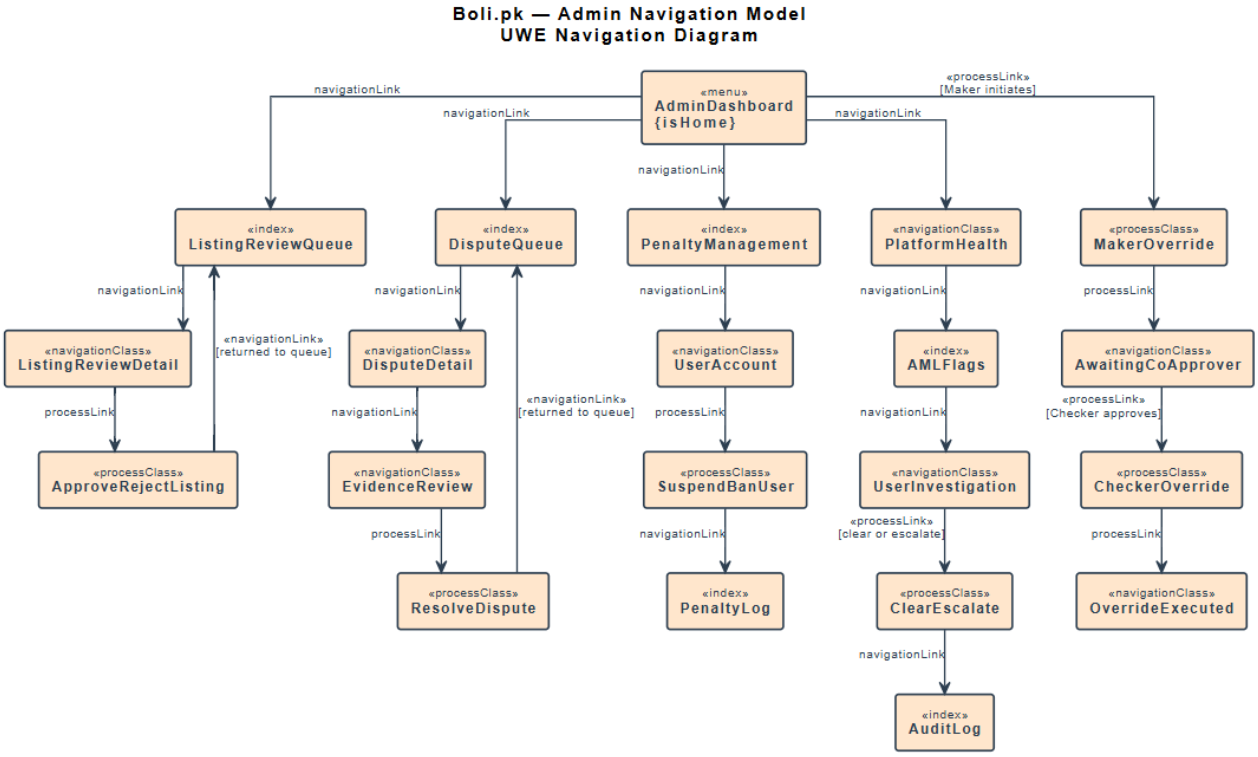
Key Navigation Paths for Seller Flow:

- Dashboard → Create Listing → [NTN guard for >Rs.100k — processLink] → Listing Form → Image Upload → [AI vetting — processLink] → Listing Preview → Auction Schedule
- Active Auctions → Live Auction Monitor → [auction close] → Escrow Confirmed → Meetup Coordination
- Meetup Coordination → [buyer IMEI scan complete] → [biometric gate — processLink] → QR Generator → [QR scanned — processLink] → Funds Received → Dashboard

- Auction Monitor → [AI quarantine] → Admin Review Notification → Dashboard
- Dashboard → My Listings → Listing Detail → Relist (if UNSOLD_EXPIRED)

8.4 Administrator Navigation Model

The administrator navigation model covers the operational flows for listing review, dispute resolution, and penalty enforcement.



Key Navigation Paths for Administrator Flow:

- Admin Dashboard → Listing Review Queue → Listing Detail → [approve/reject — processLink] → Queue
- Admin Dashboard → Dispute Queue → Dispute Detail → Evidence Review → [resolution — processLink] → Dispute Queue
- Admin Dashboard → Penalty Management → User Account → [suspend/ban — processLink] → Penalty Log
- Admin Dashboard → Platform Health → AML Flags → User Investigation → [clear/escalate — processLink]
- Admin Dashboard → Manual Override → [Maker — processLink] → Awaiting Co-Approver → [Checker — processLink] → Override Executed

8.5 State-Driven Navigation Rules

Several navigation transitions are driven by system state rather than user action. These correspond to the NR requirements in SRS v2.0:

Trigger Event	Current Screen
Auction ends — user is winner	Live Auction Room
Auction ends — user is loser	Live Auction Room
Transaction enters DISPUTED state	Any settlement screen
AI assigns PENDING_REVIEW to listing	Public gallery
Session token expires	Any secure screen
Same account logs in elsewhere	Any screen
QR settlement completes	Settlement screen
User navigates to expired auction	Marketplace

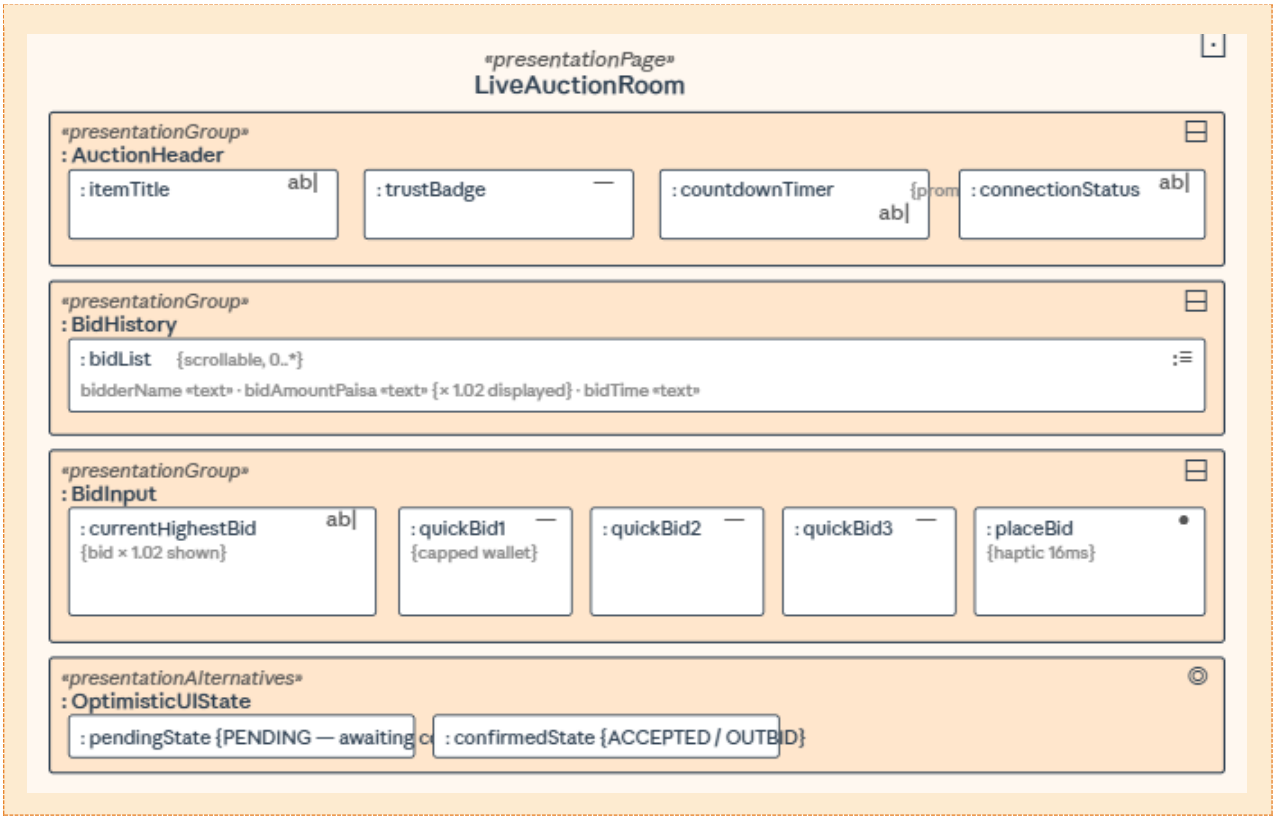
Chapter 9 Interface View — UWE Presentation Model and API Contracts

The interface view has two components. The UWE Presentation Model describes the structural layout of key screens — satisfying the Assignment 2 Presentation Model requirement and the HCI course's interface design concern. The API Contract section defines the external REST API structure — satisfying the Web Engineering and Cloud Computing course concerns for system interoperability.

9.1 UWE Presentation Model

UWE Presentation Models use abstract structural diagrams — not visual designs. They show which interface elements are present on each screen, how they are grouped, and which elements are interactive. The following screens are documented as the most critical for Boli.pk's core value proposition.

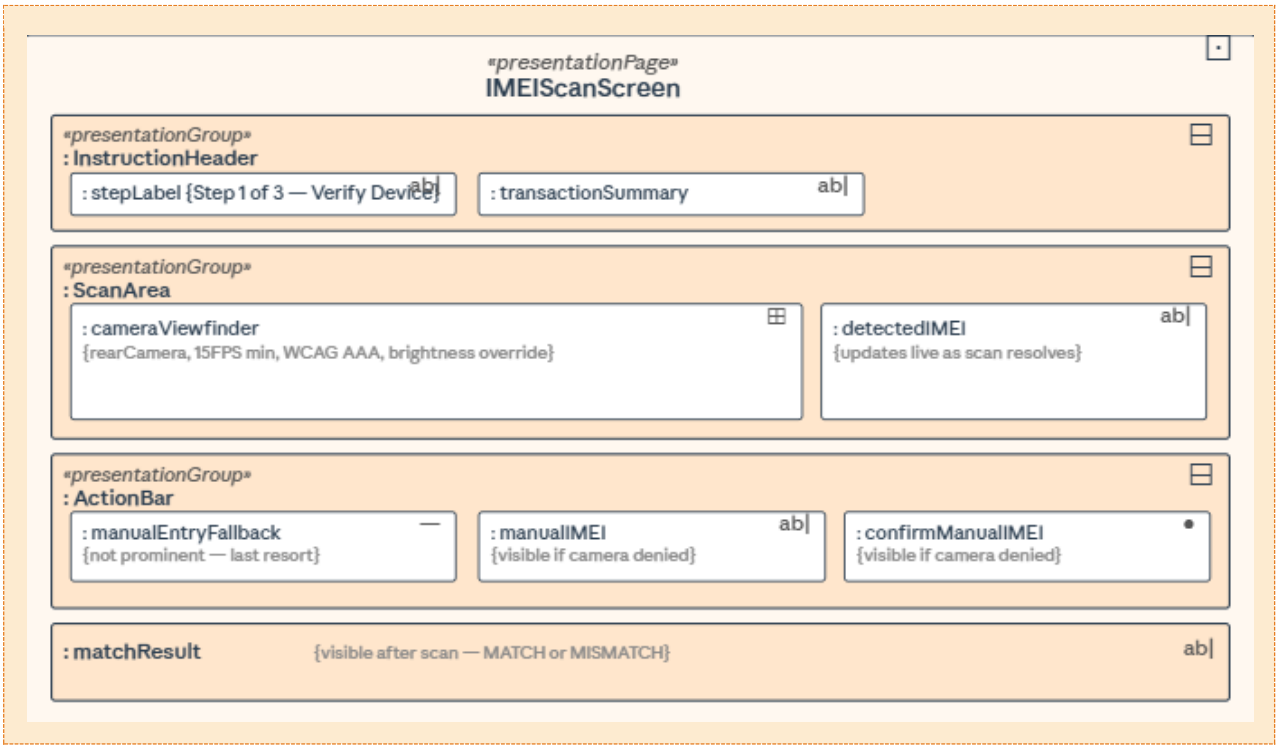
9.1.1 Live Auction Room



Design Notes:

- The current bid display must show bid × 1.02 (total buyer commitment including 2% fee) — not the raw bid value.
- Quick-bid buttons must be dynamically capped at the user's available wallet balance / 1.02.
- The countdown timer must be visually prominent — it is the primary temporal anchor for bidder decision-making.
- The Place Bid button must trigger haptic feedback within 16ms of tap before server response (IR-16).

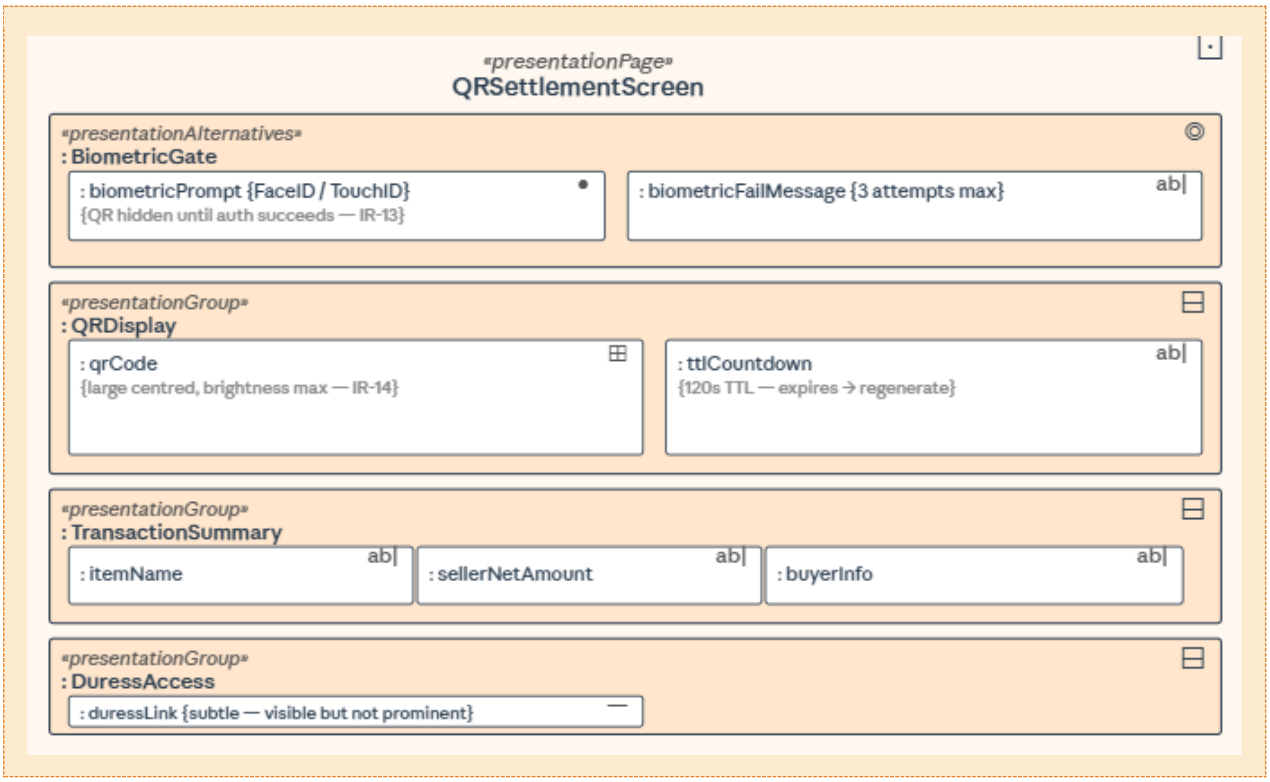
9.1.2 Meetup IMEI Scan Screen



Design Notes:

- This screen must comply with WCAG 2.1 AAA contrast (IR-18) — it is used outdoors in direct sunlight.
- The camera viewfinder must occupy maximum screen real estate to facilitate scanning.
- The step indicator (Step 1 of 3) contextualises the IMEI scan within the broader settlement flow.
- Manual entry fallback must be accessible but not prominent — it is a last resort, not the primary path.

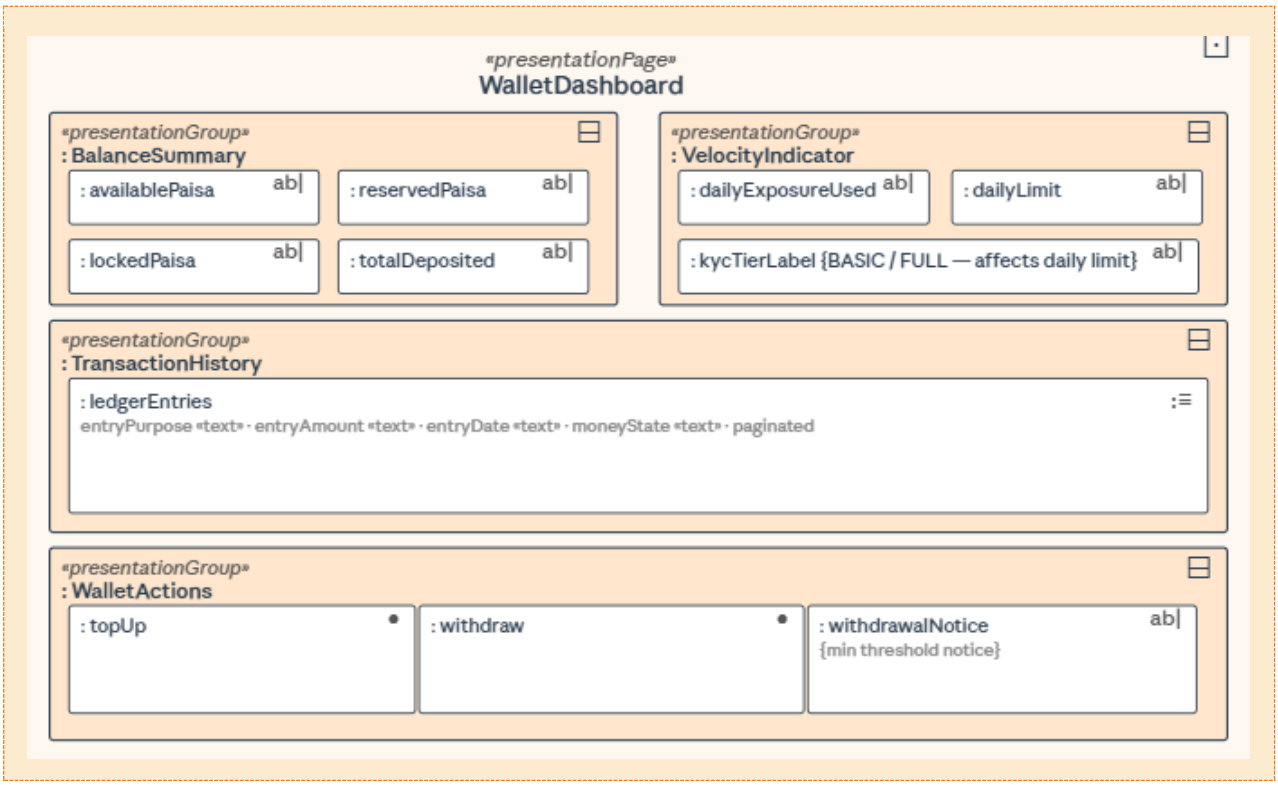
9.1.3 QR Settlement Screen — Seller



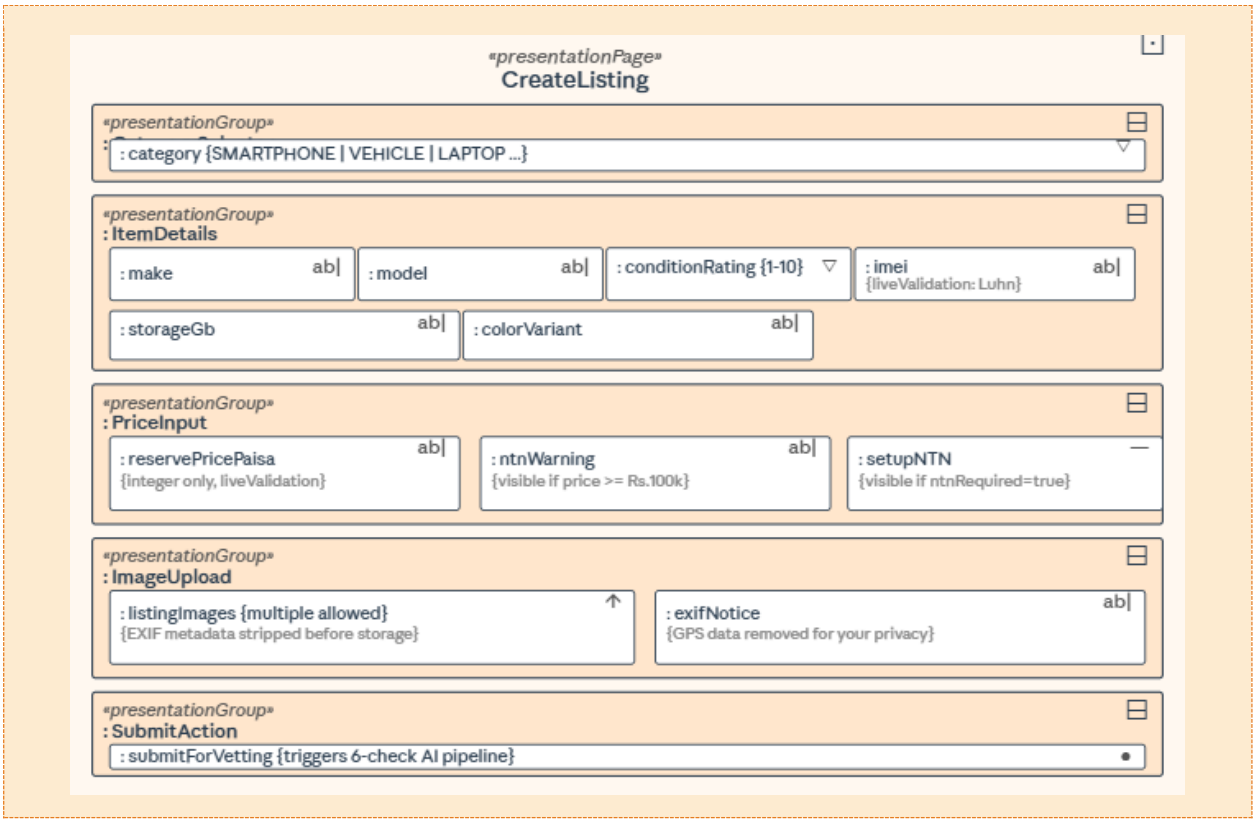
Design Notes:

- The QR code must not be visible until biometric authentication succeeds (IR-13).
- The TTL countdown is critical — the seller must know when the code expires and needs regeneration.
- The duress PIN link must be present but visually subtle — obvious enough for a coerced user but not for an attacker.
- Screen brightness must be maximised programmatically upon QR display (IR-14).

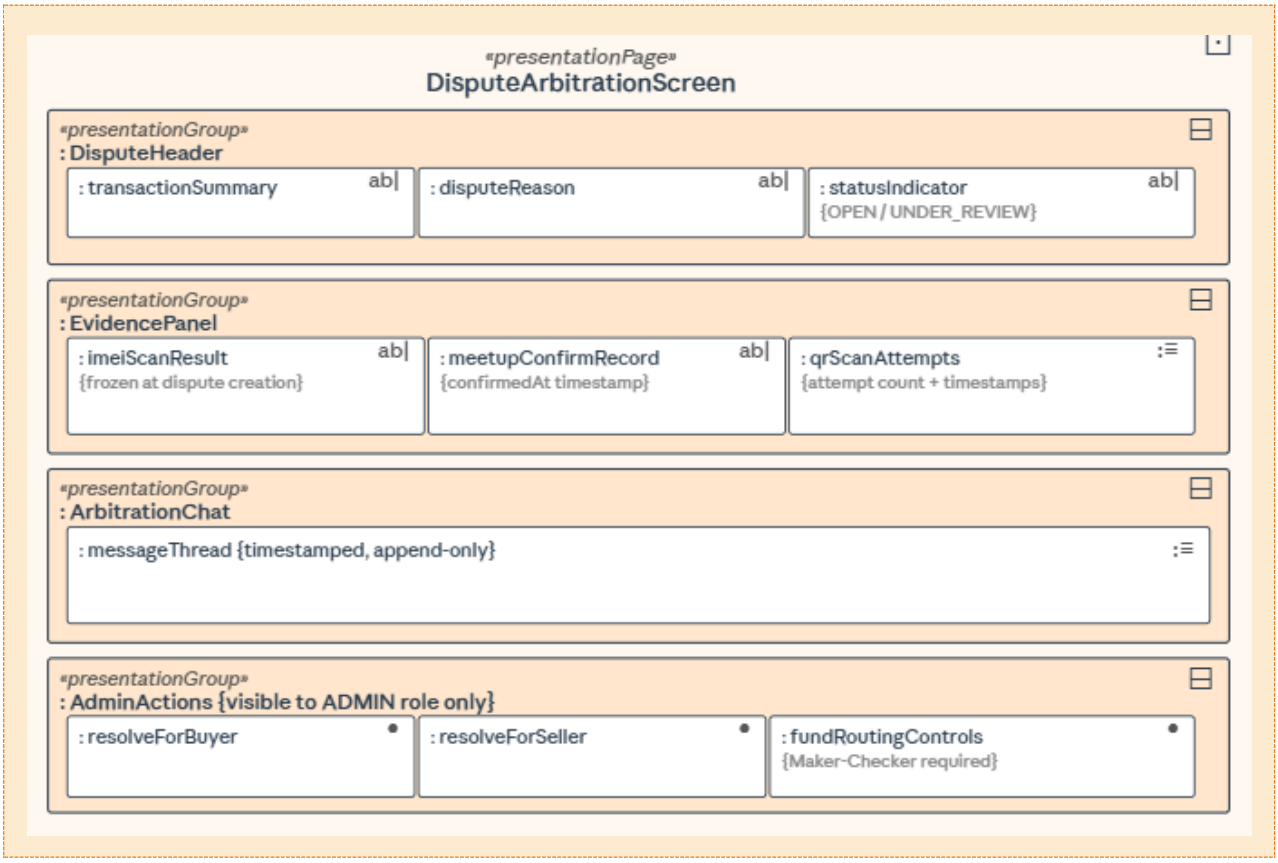
9.1.4 Wallet Dashboard



9.1.5 Listing Creation Form



9.1.6 Dispute Arbitration Screen



9.2 REST API Contract Summary

All Boli.pk REST API endpoints follow URI-based versioning (/api/v1/) and conform to OpenAPI 3.0. The following table summarises the primary endpoint groups. Full OpenAPI 3.0 YAML specifications are maintained as a separate companion document.

Endpoint Group	Base Path
Authentication	POST /api/v1/auth/
KYC/NTN	POST /api/v1/kyc/
Listings	GET/POST /api/v1/listings/
AI Vetting	POST /api/v1/vetting/
Auctions	GET/POST /api/v1/auctions/
Bids	POST /api/v1/bids/

Wallet	GET/POST /api/v1/wallet/
Escrow	POST /api/v1/escrow/
Meetup	POST /api/v1/meetup/
Settlement	POST /api/v1/settlement/
Disputes	GET/POST /api/v1/disputes/
Admin	GET/POST /api/v1/admin/
Webhooks	POST /api/v1/webhooks/

[PLACEHOLDER]

Figure 9.7 — OpenAPI 3.0 Specification Excerpt: Key endpoint definitions for escrow lock, IMEI verification, and QR settlement. Full specification maintained separately. Excerpt to be inserted here.

Chapter 10 Interaction View

The interaction view describes how system components communicate over time to accomplish key system functions. UML sequence diagrams are used to document the most critical interaction flows. This view addresses the Cloud Computing course concern for distributed system design and the SQA concern for testable component contracts.

10.1 Bid Submission Sequence

This sequence documents the complete flow from a buyer tapping Place Bid through to the bid being confirmed in PostgreSQL and broadcast to all auction participants.

[PLACEHOLDER]

Figure 10.1 — Sequence Diagram: Bid Submission Flow. Participants: Browser (Buyer), Go WebSocket Gateway, Redis Stream, Go Stream Consumer, PostgreSQL, Centrifugo, Browser (All Participants). Show: proof-of-funds check, optimistic UI response, stream publish, async consumer processing, PostgreSQL row lock, commit, broadcast. Annotate EDA boundary and idempotency key usage. To be produced using PlantUML sequence diagram and inserted here.

Key Design Points:

- The WebSocket gateway returns an immediate PENDING acknowledgement to the bidder before PostgreSQL write (Optimistic UI — IR-15).
- The Go stream consumer processes the bid asynchronously and acquires a row-level lock on the wallet before deduction (TR-04).
- On successful PostgreSQL commit, Centrifugo broadcasts the new highest bid to all participants in the auction channel.
- If the stream consumer detects a duplicate idempotency key, it returns the cached response without re-executing (TR-05).

10.2 AI Vetting Pipeline Sequence

This sequence documents the listing submission and AI vetting flow from seller image upload through to badge assignment.

[PLACEHOLDER]

Figure 10.2 — Sequence Diagram: AI Vetting Pipeline. Participants: Browser (Seller), Go Gateway, Python AI Service, Mock DIRBS Service, Mock TAC Service, PostgreSQL. Show: image upload, EXIF strip, async AI service call, three-gate sequential execution, probabilistic check execution,

badge classification, PostgreSQL listing update, seller notification. Annotate 5-second timeout and graceful degradation path. To be produced and inserted here.

Key Design Points:

- The Go gateway strips EXIF metadata before passing the image to the AI service (DR-08).
- The three hard gates execute sequentially — a gate failure triggers immediate rejection without proceeding to Phase 2.
- The full pipeline must complete within 5,000 milliseconds or the gateway triggers graceful degradation (IR-07, IR-08).
- The AI service communicates with mock DIRBS and TAC services via internal REST calls within the docker-compose network.

10.3 Two-Phase Commit Settlement Sequence

This is the most critical sequence in the entire system. It documents the complete meetup settlement flow from IMEI scan through to fund release and cryptographic receipt generation.

[PLACEHOLDER]

Figure 10.3 — Sequence Diagram: Two-Phase Commit Settlement. Participants: Browser (Buyer), Browser (Seller), Go Gateway, PostgreSQL, Redis (idempotency cache). Phase 1 (Prepare): IMEI scan result verification, biometric confirmation signal, QR seed validation, balance verification, tax pre-calculation. Phase 2 (Commit): atomic five-way ledger write (seller, WHT, ICT tax, platform revenue, buyer fee), non-repudiation receipt generation, Centrifugo settlement broadcast. Show failure path during Prepare Phase — SCAN_FAILED response. To be produced and inserted here.

Key Design Points:

- Phase 1 Prepare: all conditions verified simultaneously before any funds move. Any single failure aborts the entire settlement.
- Phase 2 Commit: all five ledger entries written in a single atomic PostgreSQL transaction. Partial commitment is impossible.
- The settlement hash (DR-12) is generated during Phase 2 incorporating buyer ID, seller ID, listing ID, verified IMEI, timestamp, and geolocation.
- The idempotency key prevents duplicate QR scans from executing a second settlement (TR-05).
- If Phase 2 fails after Phase 1 succeeds, the Saga compensating transaction reverses all Phase 1 preparations (TR-06).

10.4 Dispute Escalation Sequence

This sequence documents the flow from IMEI mismatch detection through to escrow dispute state and administrator notification.

[PLACEHOLDER]

Figure 10.4 — Sequence Diagram: IMEI Mismatch Dispute Escalation. Participants: Browser (Buyer), Go Gateway, PostgreSQL, Notification Service, Admin Dashboard. Show: IMEI scan result mismatch detection, QR unlock blocked, auto-refund initiation (S4→S11→S2), seller account flag, admin alert generation, evidence freeze. To be produced and inserted here.

10.5 End-of-Day Reconciliation Sequence

This sequence documents the automated nightly reconciliation that verifies zero-sum ledger integrity.

[PLACEHOLDER]

Figure 10.5 — Sequence Diagram: EOD Reconciliation. Participants: Scheduler, Reconciliation Service, PostgreSQL, SIEM, Engineering Alert System. Show: full ledger replay from transaction 1, running balance computation, comparison against stored balances, discrepancy detection, SIEM event emission, pager alert on non-zero discrepancy. To be produced and inserted here.

Chapter 11 Data View

The data view defines the persistent data structures, their relationships, and the integrity constraints enforced at the database layer. This view addresses the development team's concern for implementable schema design, the SQA concern for data integrity enforcement, and the FBR/SBP concern for financial record integrity.

11.1 Entity Relationship Diagram

[PLACEHOLDER]

Figure 11.1 — ERD: Complete Boli.pk database schema. Entities: users, wallets, listings, trust_badges, auctions, bids, transactions, escrows, meetup_sessions, disputes, ledger_entries, tax_accounts, penalties, notifications, meetup_confirmations, idempotency_keys. All foreign keys, cardinalities (crow's foot notation), and primary keys shown. To be produced using Mermaid.js erDiagram syntax or draw.io and inserted here.

11.2 Critical Schema Constraints

The following constraints must be enforced at the database layer, not only at the application layer:

11.2.1 Financial Integrity Constraints

- All monetary columns: INTEGER type only. NUMERIC and FLOAT types are prohibited.
- wallets table: CHECK constraint enforcing available_paisa + reserved_paisa + locked_paisa = total_deposited_paisa at all times.
- ledger_entries table: INSERT-only table. UPDATE and DELETE permissions revoked from all application roles.
- ledger_entries table: pre-INSERT trigger verifying sum of all debits and credits in the transaction block = 0.
- transactions table: money_state column is an ENUM constrained to the 13 defined states. Invalid states are rejected at the database level.

11.2.2 Hash Chain Integrity

- ledger_entries.previous_hash: SHA-256 hash of the preceding entry's data fields.
- ledger_entries.current_hash: SHA-256 hash of this entry's data fields concatenated with previous_hash.
- A background monitoring process verifies hash chain continuity every 60 seconds and emits a CHAIN_INTEGRITY_FAILURE SIEM event on detection of any break.

11.2.3 Immutability and Retention

- All financial tables: row-level security policies prevent UPDATE and DELETE for all roles except the reconciliation service.
- Retention policy: automated deletion of records created less than 5 years ago is blocked by a database trigger that raises an exception.
- Archival policy: records older than 90 days are migrated to cold storage. Primary database tables retain only the last 90 days of active records.

11.2.4 Encryption

- users.cnic, users.ntn, users.phone: encrypted at rest using AES-256-GCM with keys managed by the runtime secrets manager.
- escrows.qr_seed_encrypted: AES-256-GCM encrypted. The key is rotated per-transaction.
- biometric_hashes: one-way SHA-256 hashes. Original biometric data is never stored.

11.3 Key Table Definitions

The following table definitions are provided as reference for implementation. Full schema with all indexes and constraints is maintained in the database migration files.

11.3.1 users table

Column	Type
user_id	UUID
cnic_encrypted	BYTEA
ntn_encrypted	BYTEA
kyc_tier	ENUM(BASIC,FULL)
trust_score	SMALLINT
is_seller_suspended	BOOLEAN
suspension_until	TIMESTAMPTZ
is_buyer_banned	BOOLEAN
created_at	TIMESTAMPTZ
deleted_at	TIMESTAMPTZ
anonymised_uuid	UUID

11.3.2 wallets table

Column	Type
wallet_id	UUID
user_id	UUID
available_paisa	BIGINT
reserved_paisa	BIGINT
locked_paisa	BIGINT
total_deposited_paisa	BIGINT
CONSTRAINT balance_invariant	CHECK

11.3.3 ledger_entries table

Column	Type
entry_id	UUID
transaction_id	UUID
from_wallet_id	UUID
to_wallet_id	UUID
amount_paisa	BIGINT
entry_type	ENUM(DEBIT,CREDIT)
purpose	ENUM(...)
previous_hash_sha256	CHAR(64)
current_hash_sha256	CHAR(64)
created_at	TIMESTAMPTZ
NOTE	—

Chapter 12 Algorithm View

The algorithm view documents the key algorithms and design patterns applied in Boli.pk. This view addresses the Artificial Intelligence course's concern for ML pipeline design, the SQA concern for verifiable algorithmic correctness, and the Cloud Computing concern for distributed system patterns.

12.1 Luhn Algorithm — IMEI Format Validation

The Luhn algorithm (ISO/IEC 7812) validates that an IMEI number is structurally correct before any external lookup is performed. It is the first gate in the AI vetting pipeline and executes in constant O(n) time.

Algorithm:

- Step 1: Starting from the rightmost digit, double every second digit from right to left.
- Step 2: If the doubled value exceeds 9, subtract 9.
- Step 3: Sum all digits (original and transformed).
- Step 4: If the total modulo 10 equals 0, the IMEI is valid.

Example: IMEI 490154203237518

- Digits from right: 8,1,5,7,3,2,3,0,2,4,5,1,0,9,4
- Double alternates: 8,2,5,14→5,3,4,3,0→0,4,2,4→5,5,2→3,0,18→9,8→7,4
- Sum = 60, 60 mod 10 = 0 → VALID

[PLACEHOLDER]

Figure 12.1 — Luhn Algorithm Flowchart. To be produced and inserted here.

12.2 Two-Phase Commit Protocol — Settlement

The Two-Phase Commit protocol ensures the settlement either completes entirely or not at all. It is the core financial safety mechanism of the platform.

Phase 1 — Prepare:

- Acquire exclusive row locks on: buyer wallet, seller wallet, all three tax accounts, escrow record.
- Verify QR seed cryptographic match against escrow record.

- Verify IMEI scan result stored in MeetupSession matches listed IMEI.
- Pre-calculate all five settlement amounts: seller net, WHT, ICT tax, platform revenue, zero-sum verification.
- Verify zero-sum equation holds: total in = total out to the exact Paisa.
- If any verification fails: release all locks, return PREPARE_FAILED with specific reason code.

Phase 2 — Commit:

- Write all five ledger entries in a single BEGIN/COMMIT PostgreSQL transaction block.
- Generate SHA-256 non-repudiation receipt hash.
- Update transaction money_state to S5_SETTLED.
- Release all row locks.
- Broadcast settlement confirmation to both buyer and seller via WebSocket.
- If Phase 2 fails after Phase 1 succeeds: execute Saga compensating transactions to reverse Phase 1 state.

[PLACEHOLDER]

Figure 12.2 — 2PC Settlement Flowchart showing both Prepare and Commit phases with all failure paths and compensating transaction branches. To be produced and inserted here.

12.3 Saga Pattern — Distributed Compensating Transactions

The Saga pattern manages distributed transactions across services where a single atomic transaction is not possible. When a multi-step operation fails partway through, compensating transactions undo completed steps in reverse order.

Settlement Saga Steps and Compensations:

Step	Action
Step 1	Debit buyer platform fee from wallet
Step 2	Credit WHT to WHT Holding Account
Step 3	Credit ICT Sales Tax to ICT Holding Account
Step 4	Credit platform fee net to Revenue Account
Step 5	Credit seller net to seller wallet

If Step 3 fails, compensating transactions for Steps 2 and 1 execute in reverse order. Step 5 compensation is never needed — it is the final step.

12.4 Circuit Breaker Pattern — External Service Resilience

The circuit breaker prevents cascade failure when external services are unavailable. It monitors failure rates and temporarily stops sending requests to failing services.

Circuit Breaker States:

- CLOSED (normal): requests pass through. Failure count monitored.
- OPEN (tripped): after 5 failures in 10 seconds, circuit opens. All requests immediately return cached/queued response. No calls made to external service.
- HALF-OPEN (testing): after a configurable timeout (30 seconds default), one probe request is allowed through. If it succeeds, circuit closes. If it fails, circuit stays open.

[PLACEHOLDER]

Figure 12.3 — Circuit Breaker State Machine Diagram. To be produced and inserted here.

12.5 CQRS-Lite Pattern — Read/Write Separation

CQRS-Lite separates the read path from the write path for the bidding engine specifically. This allows the system to serve thousands of concurrent auction state reads from Redis while processing writes through PostgreSQL sequentially.

Write Path (Commands):

- Bid submitted → Gateway validates proof-of-funds from Redis cache → Writes bid event to Redis Stream → Returns PENDING to bidder
- Go worker consumes stream → Acquires PostgreSQL row lock → Commits bid → Releases lock → Updates Redis cache

Read Path (Queries):

- Any read of current highest bid, time remaining, or bid history → served from Redis cache → sub-15ms response
- Redis cache is populated by the Go worker after each successful PostgreSQL commit
- Cache TTL is set to auction duration + 60 seconds to cover edge cases during close

Consistency Boundary:

The Redis cache is the authoritative source for auction state during a live auction. At auction close, the Go worker performs a final Redis-to-PostgreSQL reconciliation to ensure the winning bid recorded in PostgreSQL matches the Redis state. This reconciliation is the handoff point and is the highest-risk operation in the bidding engine.

[PLACEHOLDER]

Figure 12.4 — CQRS-Lite Flow Diagram: Write path (EDA) and Read path (cache) with consistency boundary annotation. To be produced and inserted here.

12.6 AI Vision Model — Device Classification

Check 4 of the AI vetting pipeline uses a computer vision model to verify that the device photographed matches the stated make and model. For the CEP, a pre-trained model is used with fine-tuning. Production requires a proprietary Pakistani market dataset.

Model Architecture:

- Base model: EfficientNet-B0 or MobileNetV3 (lightweight, suitable for CPU inference in the CEP environment)
- Input: listing photos, resized to 224×224 pixels, normalized
- Output: probability distribution over smartphone make/model classes
- Confidence threshold: predictions below 0.75 confidence contribute 0 points to Check 4 score
- Classification labels: organised by manufacturer (Apple, Samsung, Xiaomi, etc.) and model generation

Training Data (Production):

- Pakistani market smartphone listings from OLX with verified IMEI cross-references
- GSMA device image database for canonical reference photos
- Minimum 500 images per model class for production-grade accuracy

CEP Implementation:

- Pre-trained MobileNetV3 from PyTorch Hub, fine-tuned on a curated subset of ~50 device classes
- Inference runs on CPU — no GPU required for CEP demonstration
- Processing target: under 3,000ms for a 5MB image on a standard laptop CPU (NFR-03)

[PLACEHOLDER]

Figure 12.5 — AI Pipeline Architecture Diagram: Six-check sequential flow with input/output specifications for each check. To be produced and inserted here.

Chapter 13 Resource View

The resource view documents how system resources — memory, CPU, network connections, and database connections — are allocated, managed, and bounded. This view addresses the Cloud Computing course's concern for infrastructure design and the development team's concern for preventing resource exhaustion under load.

13.1 Memory Allocation

Component	Memory Allocation
Go bidding engine (per node)	512MB minimum, 1GB recommended
Redis (per node)	2GB maximum
Python AI service (per instance)	1GB minimum, 2GB with model loaded
PostgreSQL (per node)	4GB recommended
React PWA (client)	No server memory. Browser-managed.

13.2 CPU Allocation

Component	CPU Profile
Go bidding engine	Low sustained CPU, spike at auction close
Redis	Single-threaded, very low CPU
Python AI service	High CPU per request (model inference)
PostgreSQL	Moderate CPU, I/O bound on ledger writes

13.3 Database Connection Management

PostgreSQL has a finite connection limit. Unmanaged connection growth is the most common cause of production database failure in high-concurrency applications.

Service	Connection Pool Size

Go bidding engine	10-20 connections
Python AI service	2-5 connections
PostgreSQL max_connections	100

13.4 Network Bandwidth Allocation

Traffic Type	Estimated Bandwidth
WebSocket bid events (1,000 concurrent auctions)	~2MB/s sustained, ~20MB/s at auction close
Listing image uploads (AI vetting)	~5MB per listing submission
Push notifications	Negligible — <1KB per notification
Database replication (PKR DC to replica)	~1MB/s sustained on active platform

13.5 CEP Resource Profile

For the CEP offline demonstration running on a development laptop, the following minimum specifications are required:

Resource	Minimum Specification
RAM	8GB available (16GB recommended for comfortable demo)
CPU	4-core modern processor (Intel i5 / Ryzen 5 or equivalent)
Storage	10GB free disk space for Docker images, PostgreSQL data, and Redis persistence files
Network	None required — all services communicate within docker-compose bridge network
Browser	Chrome 120+ or Safari 17+ for WebAuthn biometric API support

Chapter 14 Pending Diagrams and Artefacts Register

The following diagrams are required to complete this SDS. Each placeholder appears in the relevant chapter. This register provides a consolidated view for production planning.

Figure	Chapter
Figure 5.1	5
Figure 6.1	6
Figure 6.2	6
Figure 6.3	6
Figure 7.1	7
Figure 7.2	7
Figure 8.1	8
Figure 8.2	8
Figure 8.3	8
Figure 9.1	9
Figure 9.2	9
Figure 9.3	9
Figure 9.4	9
Figure 9.5	9
Figure 9.6	9
Figure 9.7	9
Figure 10.1	10
Figure 10.2	10
Figure 10.3	10
Figure 10.4	10
Figure 10.5	10
Figure 11.1	11
Figure 12.1	12
Figure 12.2	12
Figure 12.3	12
Figure 12.4	12

Figure 12.5	12

End of Software Design Specification — Boli.pk v1.0 — IEEE/ISO/IEC 1016-2009